

A Methodological Approach to Model-Driven Design and Development of Automation Systems

María Luz Alvarez, Isabel Sarachaga, Arantzazu Burgos, Elisabet Estévez, and Marga Marcos, *Member, IEEE*

Abstract—The growing complexity of industrial automation demands the adoption of software engineering principles for improving the development process of control systems. This paper presents a methodological approach to the design and development of complex automation systems relying on model-driven engineering (MDE). A benefit of this approach is the integration of methods and techniques widespread within the automation discipline with modern MDE techniques guiding the designer through the development phases. A second advantage is to add flexibility enough to adapt the steps to the needs of the system under design. Finally, the architecture presented is prepared to be adapted to methodology extensions to cover other aspects of automation systems. The framework is based on domain models that are defined through the development phases using the terminology of the automation field. Using model transformations both documentation about system analysis and design and the skeleton of software units are automatically generated. A proof-of-concept tool has been developed that has been tested on the design of medium-complexity projects to assess the impact of its use with respect to project documentation and maintenance.

Note to Practitioners—Control software development can be considered one of the challenges in automation field for achieving leadership in the future economic market. This work presents a model-driven engineering-based approach making use of both automation and software engineering methods and techniques for developing automation control systems. The framework implements the methodology for industrial automation systems (MeiA_●) for guiding developers through the development phases and generates the analysis and design documentation using domain terminology, the design documentation that involves the minimal units of design, and the program organization units in one-to-one correspondence with the minimal units of design. From a practical point of view, it should be highly emphasized that developers of automation projects benefit from more structured designs, reduced number of errors, and improved project documentation.

Manuscript received March 23, 2016; revised April 29, 2016; accepted May 24, 2016. Date of publication June 21, 2016; date of current version January 4, 2018. This paper was recommended for publication by Associate Editor B. Vogel-Heuser and Editor H. Ding upon evaluation of the reviewers' comments. This work was supported in part by the University of the Basque Country (UPV/EHU) under Project UFI 11/28 and in part by the Ministerio de Economía y Competitividad and the Fondo Europeo de Desarrollo Regional under Project DPI2015-68602-R.

M. L. Alvarez, I. Sarachaga, A. Burgos, and M. Marcos are with the Department of Automatic Control and Systems Engineering, Escuela de Ingeniería de Bilbao 48013, University of the Basque Country, Bilbao 48013, Spain (e-mail: marialuz.alvarez@ehu.es; isabel.sarachaga@ehu.es; arantzazu.burgos@ehu.es; marga.marcos@ehu.es).

E. Estévez is with the Department of Electronic and Automatic Engineering, University of Jaén, Jaén 23071, Spain (e-mail: eestevez@ujaen.es).

This paper has supplementary downloadable multimedia material available at <http://ieeexplore.ieee.org> provided by the authors. The Supplementary Material contains a detailed presentation of the paper. This material is 14.6 MB in size.

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TASE.2016.2574644

Index Terms—Engineering frameworks, IEC 61131-3, industrial automation, methodology for industrial automation systems (MeiA_●), model-driven engineering (MDE), PLCopen XML.

I. INTRODUCTION

NOWADAYS, more and more complex, safe and trustworthy automation and control systems are required to support the technological change toward the factories of future concept. Reusability, flexibility, modularity, intelligence, virtual, affordable, ease to adapt, operate, and maintain, and reliability are demanding characteristics of future manufacturing factories [1]. This means that automation and control systems must automate, control, and optimize the production processes ensuring plant availability while providing high-quality production with zero defects.

Meeting these requirements needs innovation at different levels, introducing the latest advances in hardware, software, and information and communication technology domains to support the life cycle of applications. In this context, control software development can be considered one of the challenges in automation field for achieving leadership in the future economic market.

In this sense, recent works confirm this need. Basile *et al.* [2] propose the introduction of new software and methodological tools yet available from industry and research to build control applications that meet the new challenges.

Dubey [3] presents a critical inspection on the state-of-the-art practices related to software engineering, methods, and tools used in the development life cycle of automation applications. This paper concludes that the use of software engineering tools and techniques is still at an early stage, and points out the need for analysis and design methodologies as well as support tools that abstract the user from the particular underlying technologies. The need for methodologies and tools is also suggested in [4], after describing the challenges in adopting software engineering principles with respect to requirement elicitation, static analysis, metrics, and version management. Mainly because adapting methodologies and tools from the software engineering domain implies a great effort is not always efficient. A more detailed work [5] reviews the software engineering approaches used in the automation domain based on the taxonomy of software engineering defined by Software Engineering Body of Knowledge [6]. Among other open issues for future research, the need for a new generation of software tools that help software engineers in the use of formal methods is highlighted.

A review of modeling methodologies and technologies for requirement analysis and architectural/detailed design stages is described in [7]. A formal specification of requirements is demanded for automation applications, as current engineering tools fail to deal with requirement analysis and system design stages.

Vogel-Heuser *et al.* [8] discuss in detail the different phases along the life cycle in order to identify their specific constraints and derive specific challenges.

Consequently, the adoption of software engineering principles in industrial automation requires methodologies and tools that cover more than one development phase, ensuring discipline-specific views or, at least, support for method/tool integration and collaboration. It is also important offering support for a flexible specification using multiple projections [9] in order to avoid misunderstanding during the specification process. The essential challenges of software engineering and the requirements that software has to fulfill in the domain of automation are introduced in [10], taking into account the functional characteristics, specific constraints, and circumstances for deriving requirements concerning usability, the technical process, the automation functions, used platform, and the well-established models.

However, in order to cope with the growing complexity of software, abstraction levels must be introduced during the development cycle, as the model-driven engineering (MDE) paradigm proposes. MDE technologies combine domain-specific modeling abstractions (described using metamodels expressing domain ontologies) with transformation engines and generators, which allows defining the application structure, behavior, and requirements within particular domains, and transformation engines and generators that process certain aspects of models and synthesize various types of artifacts [11]. Vogel-Heuser *et al.* [8] discuss the automated production system from an MDE point of view highlighting the challenges, discussing the state-of-the-art methodologies and giving a roadmap for future development.

MDE can be seen as a trend in industrial automation research, since in the last years, many approaches have been proposed in the literature offering integrated development environments based on model-driven software engineering.

The most relevant ones [12]–[18] have been analyzed attending to the following criteria:

- 1) existence of methodologies for designing and developing the control application;
- 2) support for application life cycle;
- 3) technologies, methods, and standards used;
- 4) consideration of operation modes;
- 5) degree of automation of the development process;
- 6) support for the documentation process.

Most of these works consider the requirement specification to define the system behavior, but without providing methodologies for requirement elicitation and operation mode definition. Most of them only consider the automatic operation mode, except ProcGraph [16] that takes into account the operation modes from the S88 standard.

Some approaches propose analysis and design methodologies [14], [15], [18] that establish the steps to

develop the control application, but providing directives and guidelines closely linked to the technology used. Most approaches employ Unified Modeling Language (UML) diagrams and profiles for describing domain models. However, some authors [19]–[22] consider Systems Modeling Language (SysML) as a possible alternative to UML, but do not provide integrated development environments except [18] and [22]. Vogel-Heuser [23] summarizes results of usability experiments evaluating UML and SysML as software engineering notations for MDE applied in the domain of manufacturing systems.

All these works use the International Electrotechnical Commission (IEC) 61131 standard or the IEC 61499 standard as implementation target in order to assure portability of applications. Related to documentation, these approaches do not pay special attention to the documentation that must be generated in each phase of the development process.

The technologies used in those works are far from the problem domain, leading to a poor industrial acceptance because they require expertise on software engineering practices and/or UML or SysML background [24]. An alternative could be the definition of domain-specific modeling languages and tools [25]–[27] using methods and standards widely used in the automation field and abstract the management of such models as the MDE promotes. In this context, GRAphe Fonctionnel de Commande, Etapes, Transitions (GRAFCET) [28] is a widespread modeling language. On the other hand, different guidelines for the definition and handling of operation modes are available, the most popular being analyzed in [29]: Guide des Modes d'Etude et d'Arrêts Marches (GEMMA), S88 standard from International Society of Automation, and Packaging Machine Language (PackML) proposed by Open Modular Architecture Controls users group.

The goal of this paper is to validate the MDE approach using automation methods. In particular, GRAFCET and GEMMA [30] are used, although the basic ideas from S88 and PackML are also incorporated. GEMMA constitutes the natural extension of GRAFCET for defining operation modes. Several authors have developed different methodologies combining these graphical formalisms. Ponsa *et al.* [31] propose the implementation of the production cycle with GRAFCET, which is the starting point for the application of GEMMA in order to develop automation systems. González *et al.* [32] combine UML use cases, GEMMA, and GRAFCET in an object-oriented methodology to systematize the analysis process for discrete event sequential systems. Machado and Seabra [33] translate the GEMMA associated with the system behavior to a high-level sequential function chart (SFC) [34] and each GEMMA state is translated to low-level SFCs, achieving the synchronization by means of the vertical coordination. Nevertheless, none of these GEMMA- and GRAFCET-based methodologies profits from the advantages of the MDE.

As an attempt to solve these gaps, this paper presents an MDE-based approach making use of both automation and software engineering methods and techniques giving full support to the developer of automation control systems. The framework implements the methodology for industrial automation systems (MeiA_•) [35] that guides developers

through the development phases, from analysis and design to implementation and operation. Therefore, the framework provides the benefits of the synergy between MeiA methodology application and the use of MDE technologies for defining domain ontologies and model transformations.

As an abstraction layer for the final user, this framework provides an intuitive graphical interface that allows an easy and graphical application development of production automation applications. Furthermore, it generates the analysis and design documentation using domain terminology. The analysis documentation consists of GEMMA and UML-like use case diagrams [36] which describe the functional and nonfunctional requirements, although it would be possible to generate this documentation according to S88 (from level 0 to 3) or PackML. The design documentation involves the minimal units of design, the so-called design organization units (DOUs), expressed as a GRAFCET and needed to generate the program organization units (POUs) in one-to-one correspondence. The latter are generated in PLCopen XML format [37] using an IEC 61131-3 SFC code generator [38].

This paper is structured as follows. Section II describes the general architecture of the framework and presents the design of the framework modules that are in charge of implementing the MeiA methodology, the transformers that generate the internal models, and the generators that automate the documentation output and the code. Section III illustrates, as a proof of concept, a framework prototype illustrated through a case study that consists of a mechatronic station. A performance assessment is presented in Section IV, while Section V gives some concluding remarks and future work.

II. MeiA FRAMEWORK ARCHITECTURE

The MeiA framework supports the development of industrial automation system according to the MeiA methodology [35]. Based on the user requirements, the six phases of MeiA manage the analysis and design of the system from different perspectives. Each perspective is related to one operation mode that refers to the corresponding MeiA phase (Main Sequence, Manual Operation mode, Test Operation mode, Failures, Emergency, and Normal Production). Each phase comprises a set of steps that allows defining the industrial control system using design guidelines.

During the analysis of each perspective, the framework supports the generation of GEMMA and use case diagrams for identifying the control and monitoring requirements, as well as the configuration of the operator panel and other required auxiliary panels.

During the design, the framework supports the generation of the GRAFCETs in charge of organizing and coordinating the possible system states (these are called *decision* DOUs) as well as the control signals to be used in the GRAFCETs related to the actions of the production cycle (namely, the *production* DOUs). It also establishes a guideline for coordinating different tasks: the vertical coordination (also called hierarchical coordination) is used for decision DOUs, while both horizontal and vertical coordinations are used for production DOUs.

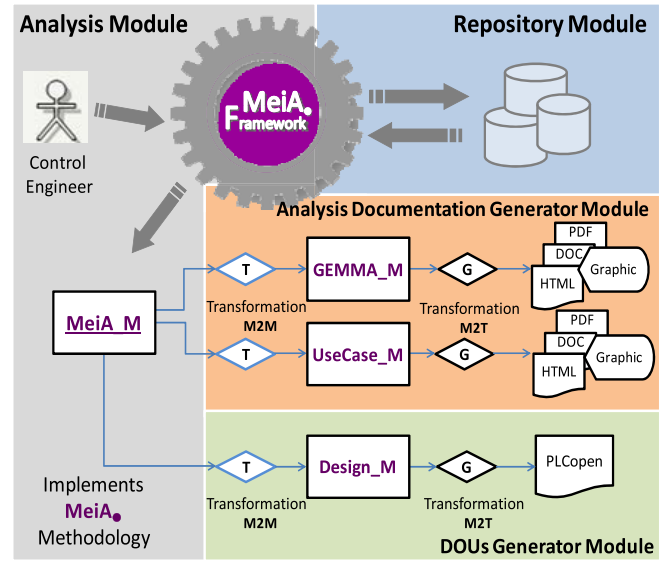


Fig. 1. MeiA framework architecture.

Based on MDE principles, the MeiA framework must also satisfy the following requirements in order to support the development of automation applications:

- 1) a user interface offering an easy, intuitive, and graphical guide to application design, following the MeiA methodology;
- 2) an incremental development process to support multidisciplinary developing team through the analysis, design, and implementation phases;
- 3) the generation of documentation for the analysis and design phases using domain terminology, in this case, the GEMMA guide, the use case diagram containing use case descriptions, and the DOUs;
- 4) the generation of code skeleton of POUs in PLCopen XML format from the DOUs;
- 5) the storage of the artifacts generated in the development process in distributed repositories for later management.

Fig. 1 illustrates the framework architecture. The core module is the *Analysis module* that implements the MeiA methodology. It interacts with the developer by means of different menus and perspectives that manage the analysis and design of the different operation modes of the system. As a result, the MeiA model is generated. From the MeiA model, different transformers generate domain models from which documentation and/or code are generated.

The *Analysis Documentation Generator module* comprises two transformers: one generates the GEMMA model and the other one generates the UseCase model. These two models follow their corresponding metamodels described in the following sections. From GEMMA and UseCase models, the analysis documentation is generated.

The *DOUs Generator module* consists of a transformer that generates the Design model containing the DOUs. From these DOUs, the skeleton of the POUs is automatically generated on a one-to-one basis.

The four domain models (MeiA_M, GEMMA_M, UseCase_M and Design_M), illustrated in Fig. 1, are stored

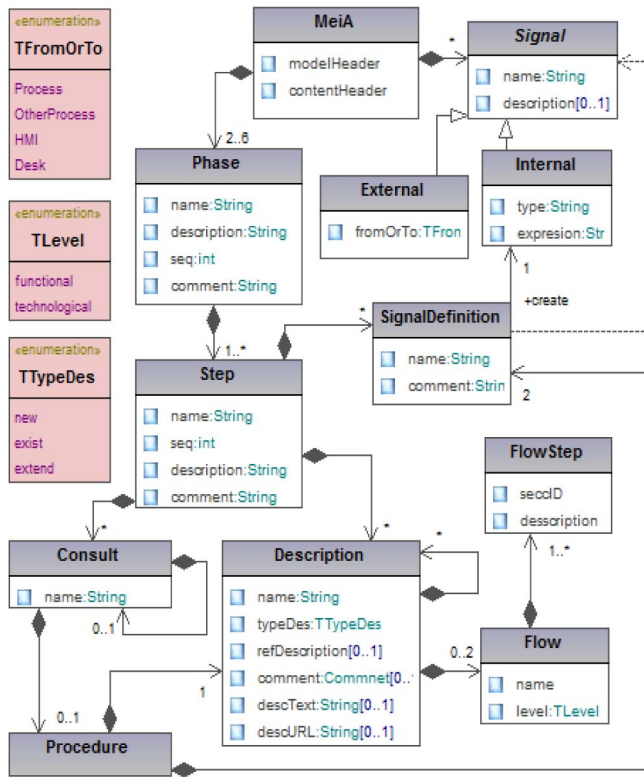


Fig. 2. MeiA_MM metamodel.

in distributed repositories for further management, as presented in [39].

The following sections present the domain ontologies for the four domains. Each metamodel describes the concepts of the domain, the relationships between them, and the structuring rules that constrain the model elements and combinations in order to assure correct domain descriptions. Section II-E introduces the transformations to automate the generation of development artifacts.

A. MeiA Metamodel

The MeiA Metamodel illustrated in Fig. 2 represents the abstract concepts of the MeiA methodology, as well as their relationships. It represents the information collected through the analysis and design of an automation system from different perspectives. The concepts and their meaning are detailed in [35].

A MeiA model comprises a maximum of six phases that allow analyzing the system from different points of view. The *first phase (Main Sequence)* analyzes the control system startup in automatic operation mode, the safe stop, and the coordination signals. These signals inform the production procedures if the system is ready to start the production or a stop at the end of cycle has been requested. The *second phase (Manual)* analyzes the manual operation mode to verify certain individual movements or process parts out of the usual cycle sequence. The *third phase* analyzes the need to verify certain movements (or process parts) either step by step or continuously, but according to the usual cycle sequence. The *fourth phase (Failures)* analyzes the process failures for achieving the diagnosis and the actions to recover

every failure. The *fifth phase (Emergency)* analyzes the emergencies and the actions to take the process to a safe state. Finally, the *sixth phase (Normal Production)* analyzes the process actions for defining the production cycle.

A phase consists of a variable number of steps depending on the application requirements. For instance, Phase 1, Main Sequence, consists of a maximum of six steps where Steps 1, 4, and 5 are mandatory and Steps 2, 3, and 6 are optional (Table I). Mandatory steps are characterized by a *SignalDefinition* in the model. This is a Boolean signal with the value of an expression that describes the start and the end of an operation mode. For instance, *ProcessID_ReqAutoMode* describes the start of the automatic operation mode. The naming procedure followed is *ProcessID_ReqStepID*. Optional steps are created if the user answer to a question is true (Consult), e.g., Phase 1 Steps 2, 3, and 6 Consult, *PIS{2, 3, 6}Consult*. This implies the need for a procedure (*ProcessID_StepIDProc*) and two new signals (*ProcessID_ReqStepID* and *ProcessID_EndStepID*) that indicate when the procedure starts and ends, respectively.

Taking all this information into account, the Phase concept in MeiA_MM is characterized by its name, a short description, the number in the phase sequence, and additional information. A phase is structured in steps that specify the analysis tasks to be performed.

The Step element is characterized by a name, the number in the step sequence, a short description, and additional information. A Step is composed by a combination of three analysis elements: *Signal Definition*, *Procedure Description*, and *Consult* elements. These three analysis elements are characterized by a name, a short description, and additional information.

The *Signal Definition* element analyzes the start and end of operation modes, procedures, and coordination tasks in order to create *Internal* signals for coordination purposes. This definition may involve two types of signals: 1) *internal* and 2) *external* (coming from/going to operator panel, process, Supervisory Control And Data Acquisition System (SCADA)...).

The *Procedure Description* element analyzes a procedure for defining its functionality at different levels of detail: from a general textual description to a detailed description in terms of the steps to be done for performing the procedure. This is the goal of the *Flow* and *FlowSteps* elements. Up to two Flow elements can be defined depending on the level of detail of the *FlowSteps* (characterized by the sequence number and the description): one for functional description, performed during the analysis, and another performed during the design, containing the technological description.

The *Consult* element checks if an analysis task is required. If so, it contains either another *Consult* or a *Procedure* that is composed of one *Procedure Description* element and two *Signal Definition* elements.

The correct definition of the MeiA model of a system (*MeiA_M*) requires the application of a set of composition rules that implements the MeiA methodology [35], deciding the optional phases and optional steps that apply to such system. The complete set of composition rules is derived from the flow diagram of the different MeiA phases.

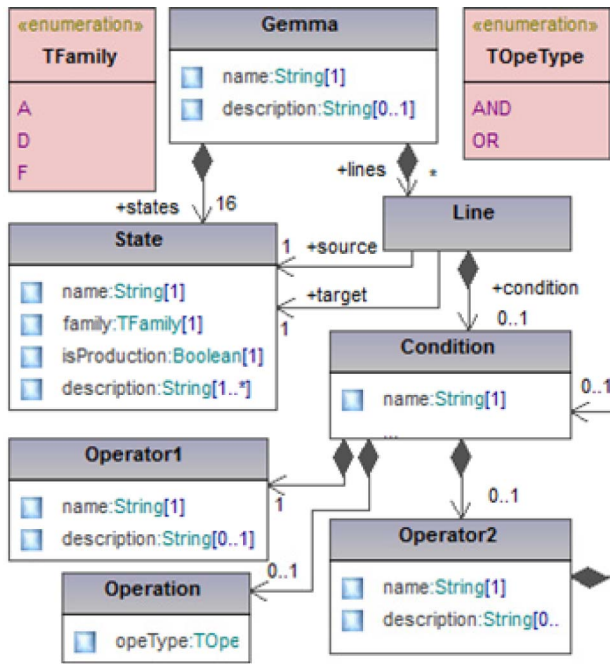


Fig. 5. GEMMA_MM metamodel.

- 1) the information for generating the analysis documentation related to GEMMA guide;
- 2) the start and stop modes (states) of an automated process from a control perspective, the transitions between states and conditions for such transitions.

As far as the authors know, a metamodel for GEMMA has not yet been proposed. Fig. 5 illustrates a metamodel that follows the guidelines established by Agence nationale pour le développement de la Productique Appliquée à l'industrie (ADEPA).

A GEMMA diagram consists of a maximum of sixteen States grouped in families that represent the start and stop modes, and oriented Lines that indicate the transition between states according to Conditions. A Condition consists of one expression (Operator) or more than one joined by logical operations (Operation).

The composition rules perform the semantic domain constraints: the source and destination states of an oriented Line must be different states; a state without incoming Lines cannot be active; and the source and destination states of an oriented Line must be active states.

D. DOU Description Metamodel

The *Design_M* model contains all information required to define and represent the minimal units of the design for generating the software model of a particular control system. It must contain the following:

- 1) the information for generating the design documentation related to DOUs;
- 2) the design of functional requirements by means of step sequences (flows) in order to guide the DOU generation;
- 3) incremental DOU descriptions from a functional level to a technological level;

- 4) the information for obtaining POUs in SFC following PLCopen XML format [37] from technological DOUs.

Some references have been found that propose meta-models for GRAFCET [46], [47]. But in order to fulfill the fourth requirement, automatic generation of POUs in PLCopen XML format, a markup language expressing the concepts of GRAFCET, should be recommendable. In this sense, the intermediate modeling layer (IML) [48] of AutomationML successfully meets the four requirements, as it addresses the representation of all GRAFCET elements as well as SFC elements. Furthermore, the SFC representation in PLCopen comprises the information about the logic designs and their graphical representations, and the treatment of the graphical information requires complex algorithms that AutomationML has already solved. Consequently, IML has been adopted as the Design metamodel (Design_MM). The POU skeleton generation in PLCopen XML format is achieved by means of the IML to PLCopen XML generator [38]. However, other generators of POUs in PLCopen XML format could be used [49].

E. Automating the Generation of Developing Artifacts

The development artifacts are generated by means of transformation engines and generators that process certain aspects of the domain models for synthesizing different artifacts. Two types of transformations are distinguished: model-to-model (M2M) and model-to-text (M2T) transformations [50].

On the one hand, M2M transformations are applied to generate the analysis models (*GEMMA_M*, *UseCase_M*) and the design model (*Design_M*) from the MeIA model (*MeiA_M*), as illustrated in Fig. 1. These M2M transformations are defined using a set of transformation rules and templates. The transformation rules are applied to the MeIA model (*MeiA_M*) conforming to its metamodel (*MeiA_MM*) and a destination model (*GEMMA_M*, *UseCase_M* or *Design_M*) is generated that conforms to the corresponding metamodel. For this purpose, the transformation rules assign a value to the parameters of the template models.

For example, a GEMMA model defines a set of states and lines (transitions between states) that may have a condition. Hence, a transformation rule for generating every concept is required. Regarding GEMMA states, they can be categorized in two main groups:

- 1) mandatory states related to mandatory steps in MeIA, e.g., A1 (Initial state stop), A2 (Requested stop at the end of cycle), and F1 (Normal Production);
- 2) optional states that are generated only if the process requires optional steps in MeIA.

For instance, in the Main Sequence phase, if the process requires Steps 2, 3 and/or 6 (see Table I), it implies the generation of GEMMA states A6 (Production reset to the initial state), F2 (Startup process), and/or F3 (Shutdown process), respectively. In order to generate the lines, five kinds of transformation rules have been identified depending on the existing steps. Finally, the generation of the line corresponding to a condition is generated from the signal definitions in the MeIA model.

In the same manner, a transformation rule has been defined for generating each concept of the UseCase model: the use

cases, the descriptions of the use cases, and the relationships between use cases. Two types of transformation rules have been identified for generating the use cases.

- 1) A use case is generated from a mandatory step of a mandatory phase or from a mandatory step of an optional phase in MeiA.
- 2) A procedure of an optional step in MeiA generates a use case.

Obviously, a similar classification is required to generate the descriptions of the use cases obtained in each case. Regarding relationships, a use case generated by a procedure of an optional step always presents an $\langle\langle\text{include}\rangle\rangle$ or $\langle\langle\text{extend}\rangle\rangle$ relationship with another use case depending on the description type of the procedure: an $\langle\langle\text{include}\rangle\rangle$ relationship in case of new or exist and an $\langle\langle\text{extend}\rangle\rangle$ relationship in case of enrich.

Finally, the design model comprises a set of GRAFCETs, called DOUs (ImlSystem element). Therefore, a set of transformation rules have been defined for generating each type of DOU and for coordinating purposes. From a phase or a set of steps in MeiA, a decision DOU or a production DOU is generated, while an auxiliary DOU is generated from a procedure in MeiA. Procedure Description generates the sequence of steps and transitions with the actions and transition conditions. The two signal definitions of the procedure perform the coordination between DOUs.

On the other hand, generators that implement M2T transformations are applied to obtain both graphical and textual documentation about system analysis and design. Besides, as commented above, the PLCopen converter framework is used to generate the POU's skeleton in PLCopen XML format.

III. FRAMEWORK PROTOTYPE FOR DEVELOPING AUTOMATION SYSTEMS

Taking into account the framework requirements described in Section II, the prototype implementation requires three main selections:

- 1) a development framework;
- 2) a standard format for the domain models and model transformations;
- 3) a database for model storage.

The technologies used to implement the MeiA framework are illustrated in Fig. 6.

Eclipse IDE [51] has been selected as the development framework. Eclipse is open source and multiplatform. It also provides a powerful visual editor, an excellent debugging engine and a flexible GUI that allows modifying the generated code without impacting on the graphical environment. But it is also an extensible plug-in system.

Since Eclipse supports multilanguage, Java and XML have been used for the framework development: Java provides portable code, while XML provides portable data. Java has been used for implementing the Analysis module, the Analysis Documentation Generator module, DOUs Generator module, and the communication interface between modules. The W3C XML schema [52] has been employed for defining the domain ontologies, and the Extensible Stylesheet Language

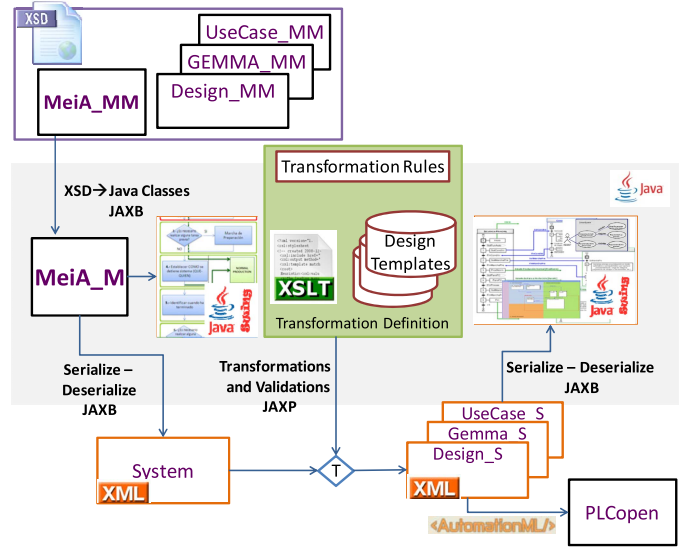


Fig. 6. Technologies used to implement the MeiA framework.

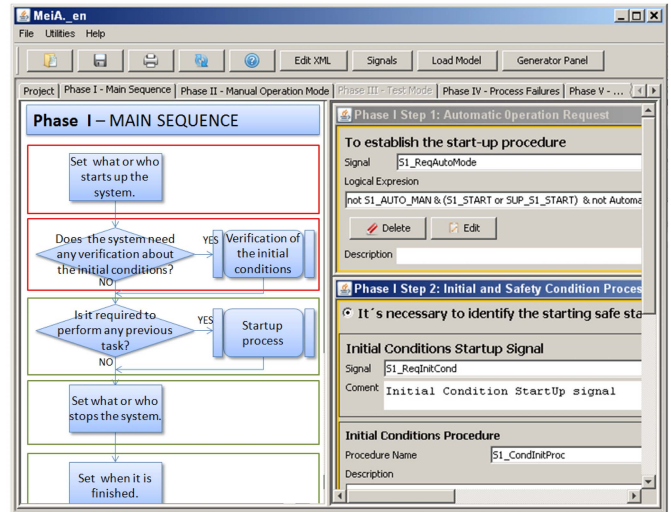


Fig. 7. Interface for Phase I-Main Sequence of the MeiA methodology.

Transformations stylesheet technology [53] for implementing M2M and M2T transformations.

Fig. 7 illustrates the GUI for the Main Sequence phase (Phase 1). It contains the flow diagram for this phase (on the left) and the data introduction for its first two steps (on the right). For every phase, the corresponding interface provides a flow diagram with the associated analysis steps. For every step, the interface provides a window with the fields that must be filled in; in this case, according to Table I. These features increase the following. 1) *learnability* as the users can learn the main system functionality easier and achieve skill to do the analysis and design quicker because they can select and analyze each operation mode using modeling languages that manage the concepts they commonly use; 2) *efficiency* as the users can perform their task faster throughout the phases related to the operation modes and the steps organized by flow diagrams; 3) *user satisfaction* as the users are more pleasant because they are guided to correctly define the industrial con-

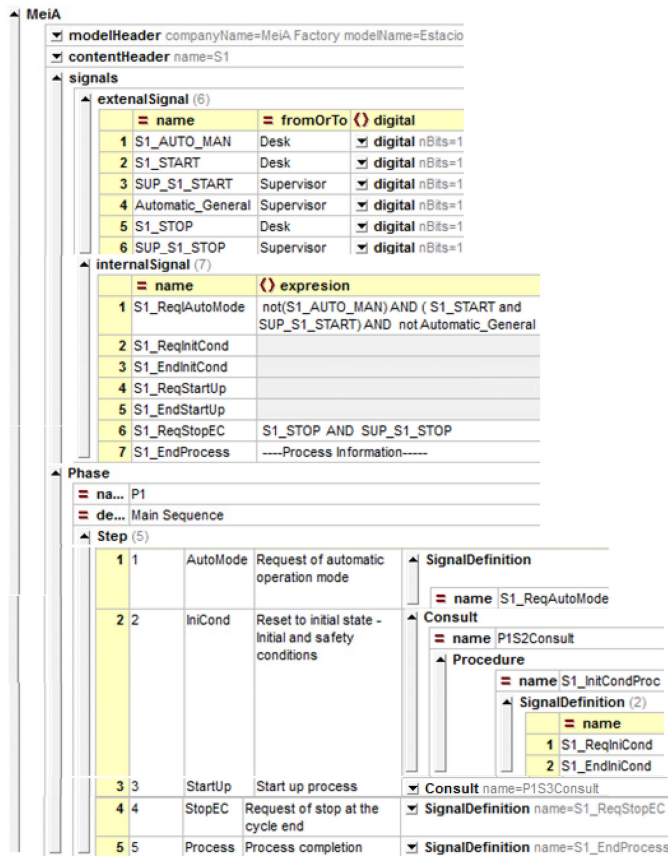


Fig. 8. MeiA_M model for Phase 1-Main Sequence.

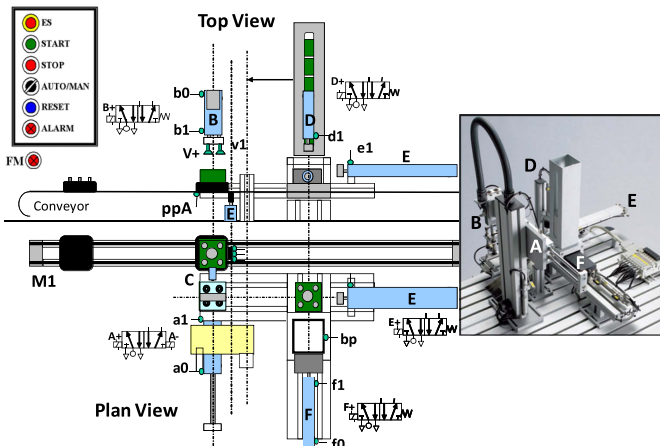


Fig. 9. Base Location—Station 1—S1.

trol system by the flow diagrams and they have the windows for data introduction, and furthermore, they get the analysis and design documentation using the vocabulary of the domain experts. Therefore, userfriendliness also increases.

A fragment of a MeiA_M model is shown in Fig. 8. This XML file stores the information of the Main Sequence phase for a process without a shutdown process (Step 6 in Table I).

To illustrate the use of the framework prototype, the development of the control software for a mechatronic station, the so-called Base Location (S1) (Fig. 9), is presented. This station

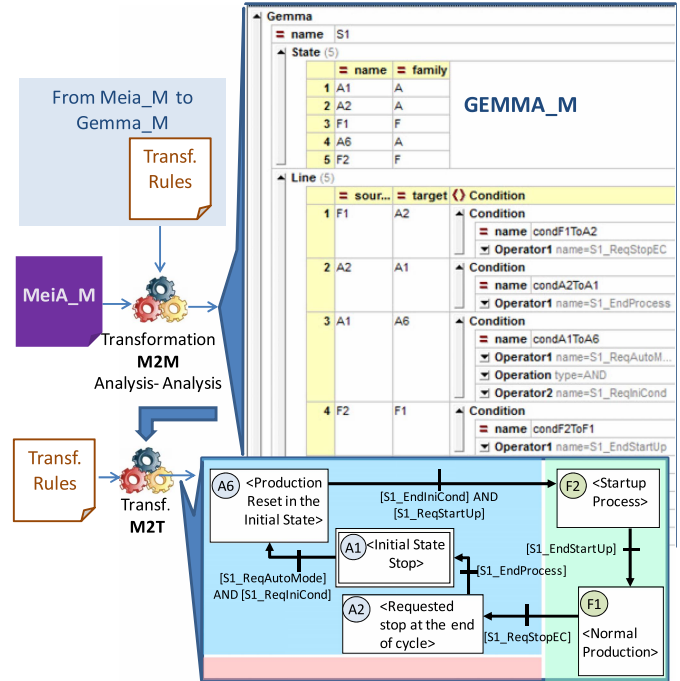


Fig. 10. GEMMA_M model and graphical representation.

is part of a flexible assembly cell FMS-200 that is in charge of mounting a set of four pieces (base, bearing, shaft and cap). Concretely, in the S1 station, the base is extracted from the base warehouse and if its position is correct, it is placed on the pallet. After that, the pallet is placed on the conveyor system and once the base is on the pallet, this is moved to the following station (Transport Station). When the position is not correct, the base is rejected.

The framework provides an intuitive graphical interface that methodologically guides developers through the development phases according to the MeiA methodology. Through the Project menu option, the analyst introduces the user requirements and selects the analysis phases to perform for the particular control system, although, if necessary, a higher detail level can be added later in the development process.

The system under design requires the following phases: Main Sequence; Manual; Failures; Emergency; and Normal Production. Each phase performs the corresponding analysis task through a set of steps, which are not independent as certain steps may require the modification or extension of the analysis results in other phase.

Fig. 7 illustrates the flow diagram for Phase I and the data introduction for the first two steps. Step 1 (request of automatic operation mode) analyzes the startup procedure of the automatic operation mode in order to define the corresponding activation signal. Step 2 (reset to initial state—initial and safety conditions) analyzes the need for a procedure to reach the starting safe state required for automatic operation mode. As the procedure is required for this example, three elements must be introduced: the signal to activate the procedure, the signal to inform the procedure end, and the procedure description. This description can be performed at different levels of detail: a descriptive text and/or a sequence of steps and/or a procedure URI.

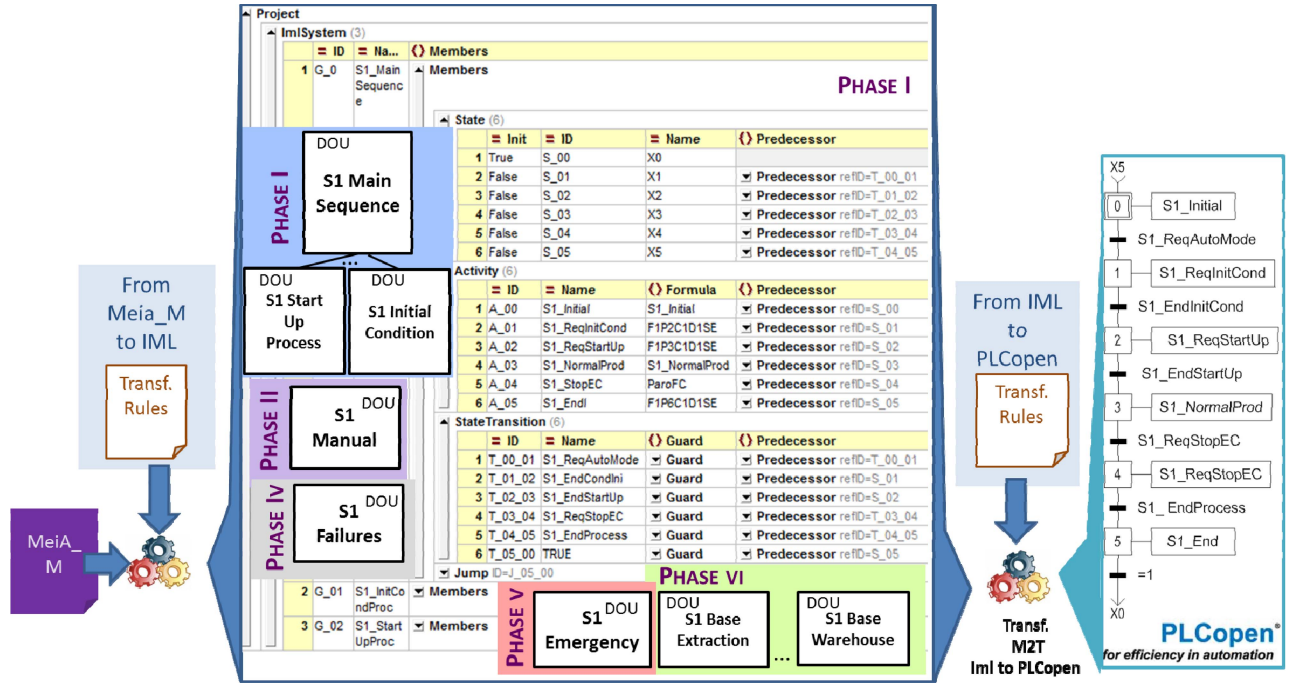


Fig. 11. Design_M model and IML file.

Once all phases have been addressed, the final result is the MeiA model that contains all the information required to design the system. Fig. 8 presents the Main Sequence fragment of the MeiA model for this case study.

From the MeiA model, the graphical and textual documentation is generated. As an example of analysis documentation generation, Fig. 10 illustrates a fragment of the *GEMMA_M* model corresponding to the Main Sequence phase. This model has been generated by means of M2M transformations, applying the transformation rules commented above (Section II-E). From this model, the graphical documentation is obtained applying M2T transformations.

Fig. 11 presents the DOUs of the control system identifying the phase that generates them. It also contains a fragment of the *Design* model generated by M2M transformations. This fragment is the DOU for the main sequence procedure of this case study. From this DOU, the design documentation is obtained applying M2T transformations and the corresponding POU is generated in PLCopen XML format using an IEC 61131-3 SFC code generator.

At this point, it is necessary to highlight that the framework is responsible for maintaining the integrity and consistency between signals and procedures.

IV. PERFORMANCE ASSESSMENT

The target of the framework is to allow methodological developments of complex automation systems. This process starts from collecting the system requirements that provide the information about how the process must operate as well as operation constraints. From this information, the different operation modes, the system states in every mode, and transitions among them can be defined. Hierarchical decomposition in subsystems and their relationships is commonly used

TABLE II
PERFORMANCE LEVELS FOR THE CRITERION OPERATION MODES

	10 (good)	..5..	1 (bad)
Analysis			
A1 <i>Operation Modes</i>	All operation modes are identified. All system states and transitions are identified	Automatic operation mode and emergency (main operation modes) are identified. Main system states and transitions are identified.	Lack of main operation modes. Lack of main system states and transitions

to cope with complex systems. Then, the detailed design, taking into account process signals, is performed and finally implementation tasks are addressed including coordination of subsystems to achieve the whole system operation.

In this section, the benefits from the use of the MeiA framework along the development cycle is validated through the application of coherent set of criteria for developer's work that includes descriptions of levels of performance quality on the criteria (a rubric) [54]. The main purpose of rubrics is to assess performances.

Fig. 12(a) illustrates the set of criteria used to assess the performance along the development phases. The set of criteria have been discussed with experts from industry involved in the design of complex applications. Each criterion is evaluated separately and the purpose is to give diagnostic information to the evaluator expert, in this case if the use of the MeiA framework improves, to some extent, the application development.

The level of performance goes from 1 (bad) to 10 (good). To illustrate this, Table II specifies the performance levels for the criterion *Operation Modes* of the Analysis Phase.

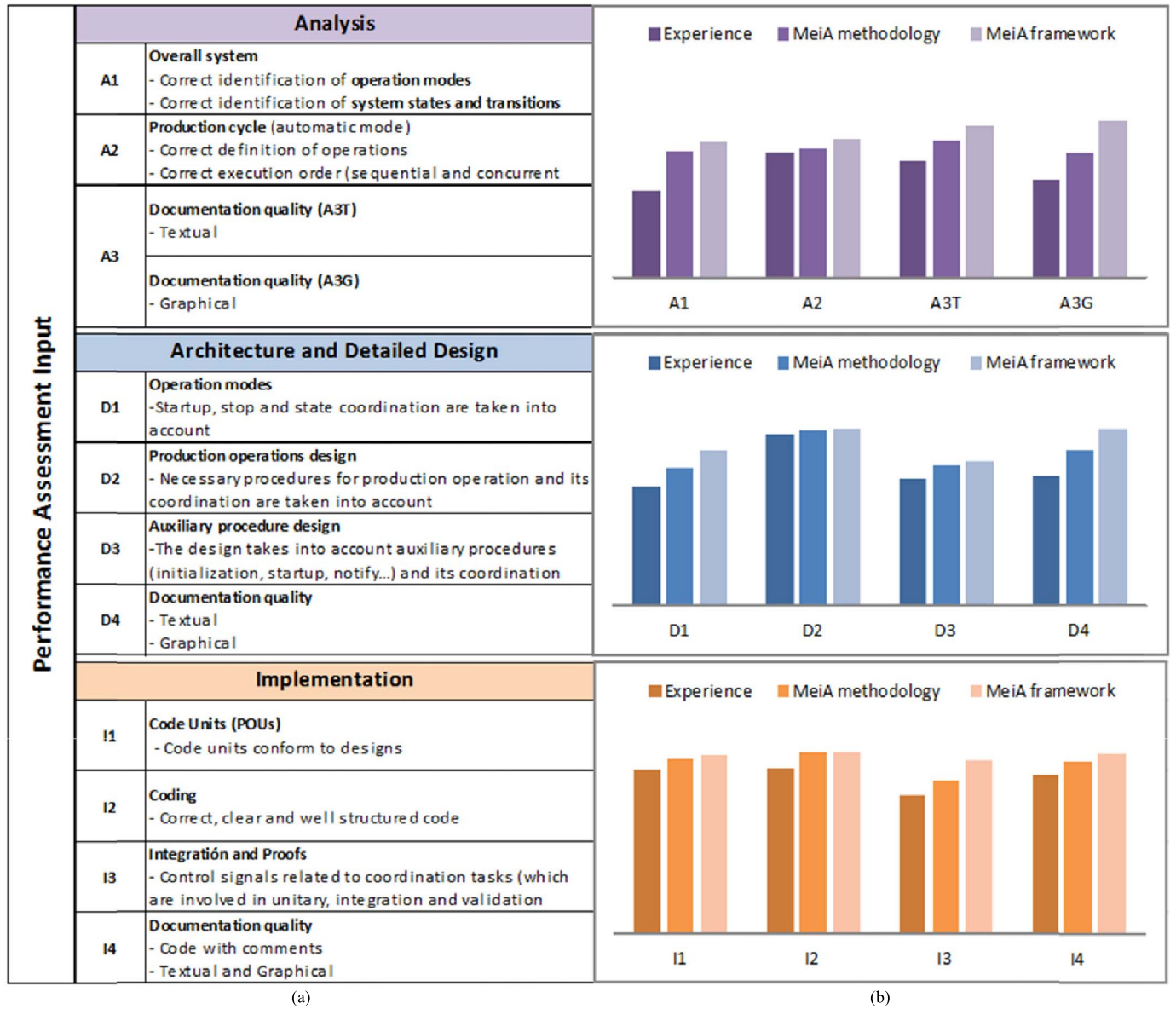


Fig. 12. (a) Evaluation criteria. (b) Result of the performance evaluation.

The performance assessment has been applied to 45 projects that deal with the development of the control system of a part of a process. As the problems tackled are rich enough, it is expected that the results can be extrapolated to bigger projects (that can be seen as sets of coordinated smaller systems).

All projects have similar characteristics in terms of complexity level (about 50 I/O signals, at least four operation modes, more than three concurrent production operations, and a maximum of 124 GRAFCET steps).

One-third of the projects have been developed from experience but not using methodological skills. Another one-third have been developed after training in the MeiA methodology, while the rest have been developed after training of the MeiA framework. All participants know the basics of automation system design according to the initial level of the training proposal that is described in [54] and [55]. This means that their skills include knowledge of GRAFCET as design tool

and PLC programming. Each developer only participated in one group.

Fig. 12(b) presents the results of the performance evaluation. Although the natural skills of the developers influence the result, the global tendencies are informative. In general, as expected from a support tool, the best indicators correspond to the use of MeiA framework. What is important to highlight is the improvement on the documentation. This is due to the automated documentation generated by the framework from the information captured along the development phases. In particular, graphical and textual documentation on system requirements as well as the mapping from requirements to design modules allow easier verification procedures.

Other key factor is on the identification of process operation modes. This is due to the procedural processes performed during system requirement identification and analysis phases. Thus, reduction of design errors and structured designs

may be expected as system operation modes are analyzed independently.

When comparing the use of MeiA methodology and the use of MeiA framework, the major improvement corresponds to the documentation generated. This is due to the potentiality of M2T transformations that are automatically performed. Furthermore, this documentation could be customized for the company.

Finally, during the implementation phase, the major improvement is achieved during integration tests, again due to cross-related information between requirements and design modules, while in the coding tasks, the improvement is not relevant as developer skills are more important.

Furthermore, although the rubric does not include development time, it is strongly related to them as development time decreases with the reduction of errors in requirement identification, the better module designs, and the better documentation. Therefore, a reduction of the time to start up the automation application can be expected, allowing shortening time to market for the developer company.

Other improvements are related to the easy extension of designs. Including a new operation mode or a control signal for coordination not only requires lower development time but also the updating of documentation is automatically performed.

Another additional but important benefit is related to the achievement of software reuse at different levels: from the minimal reuse unit to the whole automation project, as all the development artifacts are stored in distributed repositories as XML files.

V. CONCLUSION

This work has presented a novel approach that methodologically guides developers through the development phases according to MeiA methodology using methods and standards widely used in the automation field. The framework implements an incremental development process that allows different engineering profiles to work through analysis, design, and implementation phases.

This framework gives support during the design and development of the control application, guiding the user through the different phases and generating customized documentation of the overall process including the software. The modular design of the framework makes possible to adapt or extend it to fit the necessities of a company.

The performance assessment has shown the potentiality of the proposed framework to reduce design errors, provide easier verification procedures and structured designs, improve productivity, and reduce development time by means of model reuse and code generation. Other important benefits are quality improvement derived from design guidelines, validated models, and valid transformations; better documentation based on the use of models with well-defined semantics; improvement in team intercommunication as domain terminology is used; and automation project portability derived from the use of the PLCopen XML interface.

Current work is focused on the framework extension with a new phase that analyzes the information needed for achieving control system reconfiguration. This will prove

the easy extension of MeiA_MM and consequently, the easy extension of the framework, as the new phase uses the basic analysis elements (Consult, Signal Definition, and Procedure Description) used for defining a phase.

Furthermore, future works could address the framework extension with new M2M transformations in order to generate the analysis documentation for other guidelines related to the definition and handling of operation modes, such as S88 or PackML, as the MeiA model stores the information required to do it.

REFERENCES

- [1] *Factories of the Future PPP. Strategic Multi-Annual Roadmap, Adhoc Industrial Advisory Group of the Factories of the Future, Public-Private Partnership (AIAG FoF PPP), EFFRA & MANUFUTURE-EU*, Brussels, Belgium, 2010.
- [2] F. Basile, P. Chiacchio, and D. Gerbasio, "On the implementation of industrial automation systems based on PLC," *IEEE Trans. Autom. Sci. Eng.*, vol. 10, no. 4, pp. 990–1003, Oct. 2013.
- [3] A. Dubey, "Evaluating software engineering methods in the context of automation applications," in *Proc. 9th IEEE Int. Conf. Ind. Inform.*, Lisbon, Portugal, Jul. 2011, pp. 585–590.
- [4] R. Jetley, A. Nair, P. Chandrasekaran, and A. Dubey, "Applying software engineering practices for development of industrial automation applications," in *Proc. 11th IEEE Int. Conf. Ind. Inform. (INDIN)*, Bochum, Germany, Jul. 2013, pp. 558–563.
- [5] V. Vyatkin, "Software engineering in industrial automation: State-of-the-art review," *IEEE Trans. Ind. Informat.*, vol. 9, no. 3, pp. 1234–1249, Aug. 2013.
- [6] *SWEBOK: Guide to the Software Engineering Body of Knowledge*, IEEE Comput. Soc., Washington, DC, USA, 2004.
- [7] C. C. Insaurralde and A. Zoitl, "System requirements in industrial automation—A review of modeling methodologies for control software architectures," in *Proc. 11th IEEE Int. Conf. Ind. Inform. (INDIN)*, Bochum, Germany, Jul. 2013, pp. 572–577.
- [8] B. Vogel-Heuser, A. Fay, I. Schaefer, and M. Tichy, "Evolution of software in automated production systems: Challenges and research directions," *J. Syst. Softw.*, vol. 110, pp. 54–84, Dec. 2015.
- [9] K. Bengtsson and B. Lennartson, "Flexible specification of operation behavior using multiple projections," *IEEE Trans. Autom. Sci. Eng.*, vol. 11, no. 2, pp. 504–515, Apr. 2014.
- [10] B. Vogel-Heuser *et al.*, "Challenges for software engineering in automation," *J. Softw. Eng. Appl.*, vol. 7, no. 5, pp. 440–451, 2014.
- [11] D. C. Schmidt, "Guest editor's introduction: Model-driven engineering," *Computer*, vol. 39, no. 2, pp. 25–31, Feb. 2006.
- [12] C. Tranoris and K. Thramboulidis, "A tool supported engineering process for developing control applications," *Comput. Ind.*, vol. 57, no. 5, pp. 462–472, 2006.
- [13] K. Thramboulidis, D. Perdakis, and S. Kantas, "Model driven development of distributed control applications," *Int. J. Adv. Manuf. Technol.*, vol. 33, nos. 3–4, pp. 233–242, 2007.
- [14] E. Estévez, M. Marcos, I. Sarachaga, and D. Orive, "A methodology for multidisciplinary modeling of industrial control systems using UML," in *Proc. 5th IEEE Int. Conf. Ind. Inform.*, Jun. 2007, pp. 171–176.
- [15] T. Strasser *et al.*, "Multi-domain model-driven design of industrial automation and control systems," in *Proc. 13th IEEE Int. Conf. Emerg. Technol. Factory Autom.*, Sep. 2008, pp. 1067–1071.
- [16] T. Lukman, G. Godena, J. Gray, and S. Strmčnik, "Model-driven engineering of industrial process control applications," in *Proc. 15th IEEE Int. Conf. Emerg. Technol. Factory Autom.*, Sep. 2010, pp. 1–8.
- [17] D. Hästbacka, T. Vepsäläinen, and S. Kuikka, "Model-driven development of industrial process control applications," *J. Syst. Softw.*, vol. 84, no. 7, pp. 1100–1113, Jul. 2011.
- [18] B. Vogel-Heuser, D. Schütz, T. Frank, and C. Legat, "Model-driven engineering of manufacturing automation software projects—A SysML-based approach," *Mechatronics*, vol. 24, no. 7, pp. 883–896, Oct. 2014.
- [19] F. Chiron and K. Kouiss, "Design of IEC 61131-3 function blocks using SysML," in *Proc. 15th Medit. Conf. Control Autom.*, Athens, Greece, Jun. 2007, pp. 1–5.

- [20] K. Thramboulidis and G. Frey, "Towards a model-driven IEC 61131-based development process in industrial automation," *J. Softw. Eng. Appl.*, vol. 4, no. 4, pp. 217–226, 2011.
- [21] G. Barbieri, C. Fantuzzi, and R. Borsari, "A model-based design methodology for the development of mechatronic systems," *Mechatronics*, vol. 24, no. 7, pp. 833–843, 2014.
- [22] A. Fay, B. Vogel-Heuser, T. Frank, K. Eckert, T. Hadlich, and C. Diedrich, "Enhancing a model-based engineering approach for distributed manufacturing automation systems with characteristics and design patterns," *J. Syst. Softw.*, vol. 101, pp. 221–235, Mar. 2015.
- [23] B. Vogel-Heuser, "Usability experiments to evaluate UML/SysML-based model driven software engineering notations for logic control in manufacturing automation," *J. Softw. Eng. Appl.*, vol. 7, no. 11, pp. 943–973, 2014.
- [24] N. Papakonstantinou, S. Sierla, and K. Koskinen, "Object oriented extensions of IEC 61131–3 as an enabling technology of software product lines," in *Proc. 16th IEEE Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, Toulouse France, Sep. 2011, pp. 1–8.
- [25] G. Kandare, S. Strmčnik, and G. Godena, "Domain specific model-based development of software for programmable logic controllers," *Comput. Ind.*, vol. 61, no. 5, pp. 419–431, 2010.
- [26] N. Iriondo, E. Estévez, R. Priego, and M. Marcos, "Arquitectura multi-controlador con transferencia sin salto para procesos con conmutación de modos," (in Spanish), *Revista Iberoamericana Automática Informática Ind. RIAI*, vol. 10, no. 2, pp. 204–215, 2013.
- [27] N. Iriondo, E. Estévez, D. Orive, and M. Marcos, "On the use of model-based techniques for achieving multi-mode control architectures," *Mechatronics*, vol. 24, no. 7, pp. 866–882, 2014.
- [28] "AFCET (Association Française pour la Cybernétique Économique et Technique) Commission. Normalisation de la représentation du cahier des charges d'un automatisme logique," Tech. Rep., 1977.
- [29] S. Panjaitan and G. Frey, "Operation modes handling in distributed automation systems," in *Proc. 1st IFAC Workshop Dependable Control Discrete Syst. (DCDS)*, Cachan, France, 2007, pp. 109–114.
- [30] "ADEPA (Agence nationale pour le Développement de la Productique Appliquée à l'industrie). GEMMA (Guide d'Étude des Modes de Marches et d'Arrêts)," 1981.
- [31] P. Ponsa, R. Vilanova, and M. Díaz, "Introduction of the human operator into the automation cycle with the GEMMA guide," in *Proc. Int. Control Conf.*, Glasgow, Scotland, 2006, pp. 1–6.
- [32] V. M. González, F. Mateos, and A. A. C. Ng, "MLAV: The object-oriented methodology of the virtual automation lab," in *Proc. IEEE Int. Conf. Robot. Autom.*, New Orleans, LA, USA, Apr./May 2004, pp. 5153–5158.
- [33] J. M. Machado and E. Seabra, "A systematized approach to obtain dependable controllers specifications," in *Proc. ABCM Symp. Ser. Mechatronics*, vol. 4, 2010, pp. 408–417.
- [34] *GRAFCET Specification Language for Sequential Function Charts*, document IEC 60848, International Electrotechnical Commission, 2002.
- [35] M. L. Alvarez, E. Estévez, I. Sarachaga, A. Burgos, and M. Marcos, "A novel approach for supporting the development cycle of automation systems," *Int. J. Adv. Manuf. Technol.*, vol. 68, pp. 711–725, Sep. 2013.
- [36] I. Jacobson, *Object Oriented Software Engineering: A Use Case Driven Approach*. Reading, MA, USA: Addison-Wesley, 1992.
- [37] PLCopen. (2008). *XML Schemes and Documentation*. [Online]. Available: http://www.plcopen.org/pages/tc6_xml/
- [38] A. Lüder, E. Estévez, L. Hundt, and M. Marcos, "Automatic transformation of logic models within engineering of embedded mechatronic units," *Int. J. Adv. Manuf. Technol.*, vol. 54, nos. 9–12, pp. 1077–1089, 2010.
- [39] I. Sarachaga, E. Estévez, A. Armentia, A. Burgos, M. Marcos, and D. Orive, "Using Web services to support the design phase of manufacturing applications," in *Proc. 42nd CIRP Conf. Manuf. Syst.*, Grenoble, France, 2009.
- [40] OMG. (2011). *Unified Modeling Language, Superstructure Version 2.4.1*. [Online]. Available: <http://www.omg.org/spec/UML/2.4.1/Superstructure/PDF>
- [41] S. S. Somé, "A meta-model for textual use case description," *J. Object Technol.*, vol. 8, no. 7, pp. 87–106, 2009.
- [42] V. Hoffmann, H. Lichter, A. Nyßen, and A. Walter, "Towards the integration of UML- and textual use case modeling," *J. Object Technol.*, vol. 8, no. 3, pp. 85–100, 2009.
- [43] L. Zelinka and V. Vranic, "A configurable UML based use case modeling metamodel," in *Proc. 1st IEEE Eastern Eur. Conf. Eng. Comput. Syst.*, Novi Sad, Serbia, Sep. 2009, pp. 1–8.
- [44] J. Repond, P. Dugerdil, and P. Descombes, "Use-case and scenario metamodeling for automated processing in a reverse engineering tool," in *Proc. 4th India Softw. Eng. Conf.*, Thiruvananthapuram, India, 2011, pp. 135–144.
- [45] M. Misbhaudhin and M. Alshayeb, "Extending the UML use case metamodel with behavioral information to facilitate model analysis and interchange," *Softw. Syst. Model.*, vol. 14, no. 2, pp. 813–838, Mar. 2013, doi: 10.1007/s10270-013-0333-9.
- [46] F. Couffin, S. Lampérière, and J.-M. Faure, "Contribution to the Grafcet formalisation a proposal for a static meta-model," in *Proc. Simpósio Brasileiro Automação Inteligente*, São Paulo, Brazil, 1999, pp. 41–50.
- [47] F. Schumacher, S. Schröck, and A. Fay, "Tool support for an automatic transformation of GRAFCET specifications into IEC 61131-3 control code," in *Proc. IEEE 18th Conf. Emerg. Technol. Factory Autom. (ETFA)*, Cagliari, Italy, Sep. 2013, pp. 1–4.
- [48] AutomationML, "Part 4—AutomationML logic description," White Paper, 2010. [Online]. Available: <https://www.automationml.org>
- [49] F. Schumacher and A. Fay, "Formal representation of GRAFCET to automatically generate control code," *Control Eng. Pract.*, vol. 33, pp. 84–93, Dec. 2014.
- [50] K. Czarnecki and S. Helsen, "Feature-based survey of model transformation approaches," *IBM Syst. J.*, vol. 45, no. 3, pp. 621–645, 2006.
- [51] (2003). *Eclipse IDE*. [Online]. Available: <http://www.eclipse.org/>
- [52] W3C. (2004). *XML Schema Part 0: Primer Second Edition*. [Online]. Available: <http://www.w3.org/TR/2004/REC-xmlschema-0-20041028/>
- [53] W3C. (2007). *XSL Transformations (XSLT) Version 2.0*. [Online]. Available: <http://www.w3.org/TR/2007/REC-xslt20-20070123/>
- [54] S. M. Brookhart, "How to create and use rubrics for formative assessment and grading," ASCD (Asociación para la Supervisión y Desarrollo Curricular), 2013.
- [55] A. Burgos, I. Sarachaga, M. L. Alvarez, E. Estévez, and M. Marcos, "Training proposal based on MeiA to face automation challenges," *Int. J. Eng. Edu.*, vol. 30, no. 5, pp. 1254–1270, 2014.



María Luz Alvarez received the B.Sc. degree in physics, the master's degree in advanced manufacturing technology, and the Ph.D. degree in control engineering, robotic, and automatic from the University of the Basque Country, Leioa, Spain, in 1990, 1994, and 2015, respectively.

She is currently a Professor and Researcher with the Automatic Control and Systems Engineering Department with the University of the Basque Country. Since 2011, she has been a member of the GCIS Research Group (Automatic Control and Systems Engineering Department, University of the Basque Country) and acted as a Researcher on different projects related to distributed industrial control systems.



Isabel Sarachaga received the B.Sc. degree in computer science and the Ph.D. degree from the University of Deusto, Bilbao, Spain, in 1990 and 1999, respectively.

She started her work with the Robotiker Technology Centre (Mungia–Spain), where she participated as a Researcher in different communication projects related to SCADA systems, industrial networks, and fieldbuses. From 1993 to 1997, she was with the Faculty of Engineering (University of the Basque Country) as a Coordinator of the Industrial Software

Engineering Master, but also acted as a Researcher on different projects related to communications, control systems for flexible manufacturing systems, and software engineering. Since 2001, she has been a member of the GCIS Research Group and acted as a Researcher in different projects related to distributed industrial control systems. She is currently an Associate Professor of Automatic Control and Systems Engineering with the University of the Basque Country, Leioa, Spain.



Arantazu Burgos received the B.Sc. degree in computer science from the University of Deusto, Bilbao, Spain, in 1991, the master's degree in robotic and automation and the Ph.D. degree in telecommunication engineering from the University of the Basque Country, Leioa, Spain, in 1992 and 1997, respectively.

She was with the Faculty of Engineering as a Coordinator of the Advanced Manufacturing Technology Master from 1992 to 1998. Since 1992, she has been a Researcher on different projects related

to communications, industrial automation systems, software engineering, and bioelectronics, some of them in collaboration with companies in the Basque Country. Since 2006, she has also been a member of the GCIS Research Group. She is currently an Associate Professor of Automatic Control and Systems Engineering with the University of the Basque Country.



Elisabet Estévez received the B.Sc. degree in telecommunications engineering and the Ph.D. degree in automatic control from the University of the Basque Country, Leioa, Spain, in 2002 and 2007, respectively.

She is currently a Lecturer of Electronic and Automatic Engineering with the University of Jaén, Jaén, Spain. Her main expertise deals with the use of information communication technologies in service and automation industries. She has co-authored over 110 technical papers in international journals and

conference proceedings in the field of distributed industrial control systems. She has carried out review work for some conferences and technical journals.



Marga Marcos (M'88) received the Ph.D. degree in control engineering from the University of the Basque Country, Leioa, Spain, in 1988.

She was Vice Dean of the College of Engineering from 1990 to 1993. She was Chairman of the Automatic Control and Systems Engineering Department, University of the Basque Country, from 1995 to 2005, where she is currently a Full Professor. She has acted as the Main Researcher and Researcher of more than 80 research projects by National and European R&D programs. She has authored or co-

authored over 150 technical papers in international journals and conference proceedings. Recently, the focus of the research is achieving dynamic reconfiguration of distributed applications using the model driven engineering paradigm.

Dr. Marcos has served on the Technical Committees (TCs) of International Federation of Automatic Control (IFAC) and IEEE, has been a member of the European Control Association Council and the National Organizing Committee of IFAC Spain, and was the Publication Co-Chair of the IEEE Conference on Decision and Control in 2005. She was the General Co-Chair of the IEEE International Conference on Emerging Technologies for Factory Automation 2010. She is the Chair of the IFAC TC Computer for Control.